线程基础

Posted on 2023-08-06 12:17:29 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2TayNx5QxbGTaWW5s48vMjtuvCB)

简介

本文主要介绍线程的基本管控，包括线程的发起，等待，异常条件下如何等待以及后台运行等基础操作。

线程发起

线程发起顾名思义就是启动一个线程，C++11标准统一了线程操作，可以在定义一个线程变量后，该变量启动线程执行回调逻辑。如下即可发起一个线程

```
1. void thead_work1(std::string str) {
2.     std::cout << "str is " << str << std::endl;
3. }
4.
5. //1 通过()初始化并启动一个线程
6. std::thread t1(thead_work1, hellostr);
```

线程等待

当我们启动一个线程后，线程可能没有立即执行，如果在局部作用域启动了一个线程，或者main函数中，很可能子线程没运行就被回收了，回收时会调用线程的析构函数，执行terminate操作。所以为了防止主线程退出或者局部作用域结束导致子线程被析构的情况，我们可以通过join，让主线程等待子线程启动运行，子线程运行结束后主线程再运行。

```
std::string hellostr = "hello world!";
//1 通过()初始化并启动一个线程
std::thread t1(thead_work1, hellostr);
//2 主线程等待子线程退出
t1.join();
```

仿函数作为参数

当我们用仿函数作为参数传递给线程时，也可以达到启动线程执行某种操作的含义

```
class background_task {
public:
    void operator()(std::string str) {
        std::cout << "str is " << str << std::endl;
    }
};
```

如果采用如下方式启动函数，那一定会报错的。

```
std::thread t2(background_task());
t2.join();
```

因为编译器会将t2当成一个函数对象, 返回一个std::thread类型的值, 函数的参数为一个函数指针,该函数指针返回值为background_task, 参数为void。可以理解为如下

```
"std::thread (*)(background_task (*)())"
```

修改的方式很简单

```
//可多加一层()
std::thread t2((background_task()));
t2.join();

//可使用{}方式初始化
std::thread t3{ background_task() };
t3.join();
```

lambda表达式

lambda 表达式也可以作为线程的参数传递给thread

```
1.
2. std::thread t4([](std::string  str) {
3.     std::cout << "str is " << str << std::endl;
4. },  hellostr);
5.
6. t4.join();
```

线程detach

线程允许采用分离的方式在后台独自运行， `C++ concurrent programing` 书中称其为守护线程。

```
1.
2. struct func {
3.     int& _i;
4.     func(int & i): _i(i){}
5.     void operator()() {
6.         for (int i = 0; i < 3; i++) {
7.             _i = i;
8.             std::cout << "_i is " << _i << std::endl;
9.             std::this_thread::sleep_for(std::chrono::seconds(1));
10.        }
11.    }
12. };
13.
14. void oops() {
15.
16.        int some_local_state = 0;
17.        func myfunc(some_local_state);
18.        std::thread functhread(myfunc);
19.        //隐患，访问局部变量，局部变量可能会随着}结束而回收或随着主线程退出而回收
20.        functhread.detach();
21. }
22.
23. // detach 注意事项
24. oops();
25. //防止主线程退出过快，需要停顿一下，让子线程跑起来detach
26. std::this_thread::sleep_for(std::chrono::seconds(1));
```

上面的例子存在隐患，因为 `some_local_state` 是局部变量, 当 `oops` 调用结束后局部变量 `some_local_state` 就可能被释放了，而线程还在detach后台运行，容易出现崩溃。
所以当我们在线程中使用局部变量的时候可以采取几个措施解决局部变量的问题

- 通过智能指针传递参数，因为引用计数会随着赋值增加，可保证局部变量在使用期间不被释放，这也就是我们之前提到的伪闭包策略。
- 将局部变量的值作为参数传递，这么做需要局部变量有拷贝复制的功能，而且拷贝耗费空间和效率。
- 将线程运行的方式修改为join，这样能保证局部变量被释放前线程已经运行结束。但是这么做可能会影响运行逻辑。

比如下面的修改

```
1. void use_join() {
2.     int some_local_state = 0;
3.     func myfunc(some_local_state);
4.     std::thread functhread(myfunc);
5.     functhread.join();
6. }
7.
8. // join 用法
9. use_join();
```

异常处理

当我们启动一个线程后，如果主线程产生崩溃，会导致子线程也会异常退出，就是调用terminate，如果子线程在进行一些重要的操作比如将充值信息入库等，丢失这些信息是很危险的。所以常用的做法是捕获异常，并且在异常情况下保证子线程稳定运行结束后，主线程抛出异常结束运行。如下面的逻辑

```
1. void catch_exception() {
2.     int some_local_state = 0;
3.     func myfunc(some_local_state);
4.     std::thread  functhread{ myfunc };
5.     try {
6.         //本线程做一些事情,可能引发崩溃
7.         std::this_thread::sleep_for(std::chrono::seconds(1));
8.     }catch (std::exception& e) {
9.         functhread.join();
10.        throw;
11.    }
12.
13.    functhread.join();
14. }
```

但是用这种方式编码，会显得臃肿，可以采用RAII技术，保证线程对象析构的时候等待线程运行结束，回收资源。如果大家还记得我基于asio实现异步服务时，逻辑处理类LogicSystem的析构函数里等待线程退出。那我们写一个简单的线程守卫

```
class thread_guard {
private:
    std::thread& _t;
public:
    explicit thread_guard(std::thread& t):_t(t){}
    ~thread_guard() {
        //join只能调用一次
        if (_t.joinable()) {
            _t.join();
        }
    }

    thread_guard(thread_guard const&) = delete;
    thread_guard& operator=(thread_guard const&) = delete;
};
```

可以这么使用

```
void auto_guard() {
    int some_local_state = 0;
    func my_func(some_local_state);
    std::thread  t(my_func);
    thread_guard g(t);
    //本线程做一些事情
    std::cout << "auto guard finished " << std::endl;
}

auto_guard();
```

慎用隐式转换

C++中会有一些隐式转换，比如 `char*` 转换为 `string` 等。这些隐式转换在线程的调用上可能会造成崩溃问题

```
1. void danger_oops(int som_param) {
2.     char buffer[1024];
3.     sprintf(buffer, "%i", som_param);
4.     //在线程内部将char const* 转化为std::string
5.     //指针常量  char * const  指针本身不能变
6.     //常量指针  const char * 指向的内容不能变
7.     std::thread t(print_str, 3, buffer);
8.     t.detach();
9.     std::cout << "danger oops finished " << std::endl;
10. }
```

当我们定义一个线程变量 `thread t` 时，传递给这个线程的参数 `buffer` 会被保存到 `thread` 的成员变量中。
而在线程对象t内部启动并运行线程时，参数才会被传递给调用函数 `print_str` 。
而此时 `buffer` 可能随着 `}` 运行结束而释放了。
改进的方式很简单，我们将参数传递给 `thread` 时显示转换为 `string` 就可以了，
这样 `thread` 内部保存的是 `string` 类型。

```
1. void safe_oops(int some_param) {
2.     char buffer[1024];
3.     sprintf(buffer, "%i", some_param);
4.     std::thread t(print_str, 3, std::string(buffer));
5.     t.detach();
6. }
```

关于为什么参数会像我说的这样保存和调用，我在之后会按照源码给大家讲一讲。

引用参数

当线程要调用的回调函数参数为引用类型时，需要将参数显示转化为引用对象传递给线程的构造函数，
如果采用如下调用会编译失败

```
1.
2. void change_param(int& param) {
3.     param++;
4. }
5.
6. void ref_oops(int some_param) {
7.     std::cout << "before change , param is " << some_param << std::endl;
8.     //需使用引用显示转换
9.     std::thread  t2(change_param, some_param);
10.    t2.join();
11.    std::cout << "after change , param is " << some_param << std::endl;
12. }
```

即使函数 `change_param` 的参数为 `int&` 类型，我们传递给t2的构造函数为 `some_param` ,也不会达到在 `change_param` 函数内部修改关联到外部 `some_param` 的效果。因为 `some_param` 在传递给 `thread` 的构造函数后会转变为右值保存，右值传递给一个左值引用会出问题，所以编译出了问题。
改为如下调用就可以了

```
1. void ref_oops(int some_param) {
2.     std::cout << "before change , param is " << some_param << std::endl;
3.     //需使用引用显示转换
4.     std::thread  t2(change_param, std::ref(some_param));
5.     t2.join();
6.     std::cout << "after change , param is " << some_param << std::endl;
7. }
```

thread原理

为了详细说明thread参数传递和调用原理，我们看看源码

```
1. template <class _Fn, class... _Args, enable_if_t<!is_same_v<_Remove_cvref_t<_Fn>, threa
   d>, int> = 0>
2. _NODISCARD_CTOR explicit thread(_Fn&& _Fx, _Args&&... _Ax) {
3.     _Start(_STD forward<_Fn>(_Fx), _STD forward<_Args>(_Ax)...);
4. }
```

thread构造函数内部通过 `forward` 原样转换传递给 `_Start` 函数。关于原样转换的知识可以看我之前写的文章。

`_Start` 函数内部就是启动了一个线程 `_beginthreadex` 执行回调函数。

```
1. template <class _Fn, class... _Args>
2.     void _Start(_Fn&& _Fx, _Args&&... _Ax) {
3.         using _Tuple              = tuple<decay_t<_Fn>, decay_t<_Args>...>;
4.         auto _Decay_copied        = _STD make_unique<_Tuple>(_STD forward<_Fn>(_Fx), _ST
   D forward<_Args>(_Ax)...);
5.         constexpr auto _Invoker_proc = _Get_invoke<_Tuple>(make_index_sequence<1 + size
   of...(_Args)>{});
6.
7. #pragma warning(push)
8. #pragma warning(disable : 5039) // pointer or reference to potentially throwing function pas
   sed to
9.                             // extern C function under -EHc. Undefined behavior may occur
10.                            // if this function throws an exception. (/Wall)
11.        _Thr._Hnd =
12.            reinterpret_cast<void*>(_CSTD _beginthreadex(nullptr, 0, _Invoker_proc, _Decay_c
   opied.get(), 0, &_Thr._Id));
13. #pragma warning(pop)
14.
15.        if (_Thr._Hnd) { // ownership transferred to the thread
16.            (void) _Decay_copied.release();
17.        } else { // failed to start thread
18.            _Thr._Id = 0;
19.            _Throw_Cpp_error(_RESOURCE_UNAVAILABLE_TRY_AGAIN);
20.        }
21.    }
```

我们对应 `ref_oops` 内部函数的调用， `_Start` 的参数就是 `void _Start(change_param&& _Fx, int&& Ax)`

`_beginthreadex` 函数参数分别是安全参数，栈大小，调用函数地址，调用函数参数，初始标记，第三方参数地址。
我们关注 `_beginthreadex` 的调用函数和参数，调用函数为 `_Invoker_proc` ,参数为 `_Decay_copied`
我们看看这两个变量的定义

```
1. auto _Decay_copied          = _STD make_unique<_Tuple>(_STD forward<_Fn>(_Fx), _STD for
   ward<_Args>
2. constexpr auto _Invoker_proc = _Get_invoke<_Tuple>(make_index_sequence<1 + sizeof...
   (_Args)>{});
```

_Decay_copied 可以理解为

```
1. auto _Decay_copied          = _STD make_unique<_Tuple>(_STD forward<change_param>(_F
   x), _STD forward<int>
```

_Invoker_proc 可以理解为封装的可调用对象

```
1. constexpr auto _Invoker_proc = _Get_invoke<_Tuple>(make_index_sequence<1 + sizeof...(i
   nt)>{});
```

我们做一个简化

```
1. constexpr auto _Invoker_proc = _Get_invoke<_Tuple>(make_index_sequence<2>{});
```

其实就是将参数的索引0,1按照序列传递给 `_Get_invoke`
`_Get_invoke` 原型为

```
1. _NODISCARD static constexpr auto _Get_invoke(index_sequence<_Indices...>) noexcept {
2.     return &_Invoke<_Tuple, _Indices...>;
3. }
```

所以 `_Get_invoke` 函数原型为

```
1. template <class _Tuple, size_t... _Indices>
2. static unsigned int __stdcall _Invoke(void* _RawVals) noexcept /* terminates */ {
3. // adapt invoke of user's callable object to _beginthreadex's thread procedure
4. const unique_ptr<_Tuple> _FnVals(static_cast<_Tuple*>(_RawVals));
5. _Tuple& _Tup = *_FnVals;
6. _STD invoke(_STD move(_STD get<_Indices>(_Tup))...);
7. _Cnd_do_broadcast_at_thread_exit(); // TRANSITION, ABI
8. return 0;
9. }
```

所以我们可以理解为调用 `_Get_invoke` 就是调用 `invoke(_STD move(_STD`

`get<_Indices>(_Tup))...);`
`invoke(_STD move(_STD get<_Indices>(_Tup))...);` 就是将回调函数和参数传递给 `invoke`

```
1. CONSTEXPR17 auto invoke(_Callable&& _Obj, _Ty1&& _Arg1, _Types2&&... _Args2) noexce
   pt(
2.     noexcept(_Invoker1<_Callable, _Ty1>::_Call(
3.         static_cast<_Callable&&>(_Obj), static_cast<_Ty1&&>(_Arg1), static_cast<_Types2&&
   >(_Args2)...)))
```

`invoke` 实际就是调用了 `_Call` 函数, `_Call` 的作用就是调用回调函数，并传递给回调函数参数，可以理解为向change_param
传递int类型的右值数据

```
1. change_param(int&& _Arg1)
```

这与 `change_param` 的定义不符合， `change_param` 参数为左值引用， 不能绑定右值，也就是编译错误的原因。

所以在这里大家就理解了传递给 `thread` 的参数都是按照右值的方式构造为Tuple类型，传递给系统级别函数 `_beginthreadex` 调用的。
那为什么使用 `std::ref` 就可以实现引用效果呢?
这里我们看下 `std::ref` 的源码

```
1. template <class _Ty>
2. _NODISCARD _CONSTEXPR20 reference_wrapper<_Ty> ref(_Ty& _Val) noexcept {
3.     return reference_wrapper<_Ty>(_Val);
4. }
```

`reference_wrapper` 是一个类类型，说白了就是将参数的地址和类型保存起来。

```
1. _CONSTEXPR20 reference_wrapper(_Uty&& _Val) noexcept(noexcept(_Refwrap_ctor_fun<_T
   y>(_STD declval<_Uty>()))) {
2.     _Ty& _Ref = static_cast<_Uty&&>(_Val);
3.     _Ptr      = _STD addressof(_Ref);
4. }
```

当我们要使用这个类对象时，自动转化为取内部参数的地址里的数据即可，就达到了和实参关联的效果

```
_CONSTEXPR20 operator _Ty&() const noexcept {
    return *_Ptr;
}

_NODISCARD _CONSTEXPR20 _Ty& get() const noexcept {
    return *_Ptr;
}
```

所以我们可以这么理解传递给thread对象构造函数的参数，仍然作为右值被保存，
如 `ref(int)` 实际是作为 `reference_wrapper(int)` 对象保存在threa的类成员里。
而调用的时候触发了仿函数 `()` 进而获取到外部实参的地址内的数据。

绑定类成员函数

有时候我们需要绑定一个类的成员函数

```
class X
{
public:
    void do_lengthy_work() {
        std::cout << "do_lengthy_work " << std::endl;
    }
};

void bind_class_oops() {
    X my_x;
    std::thread t(&X::do_lengthy_work, &my_x);
    t.join();
}
```

这里大家注意一下，如果thread绑定的回调函数是普通函数，可以在函数前加 `&` 或者不加 `&` ，因为编译器默认将普通函数名作为函数地址，如下两种写法都正确

```
1. void thead_work1(std::string str) {
2.     std::cout << "str is " << str << std::endl;
3. }
4.
5. std::string hellostr = "hello world!";
6. //两种方式都正确
7. std::thread t1(thead_work1, hellostr);
8. std::thread t2(&thead_work1, hellostr);
```

但是如果是绑定类的成员函数，必须添加 `&`

使用move操作

有时候传递给线程的参数是独占的，所谓独占就是不支持拷贝赋值和构造，但是我们可以通过 `std::move` 的方式将参数的所有权转移给线程，如下

```
1. void deal_unique(std::unique_ptr<int> p) {
2.     std::cout << "unique ptr data is " << *p << std::endl;
3.     (*p)++;
4.
5.     std::cout << "after unique ptr data is " << *p << std::endl;
6. }
7.
8. void move_oops() {
9.     auto p = std::make_unique<int>(100);
10.    std::thread  t(deal_unique, std::move(p));
11.    t.join();
12.    //不能再使用p了，p已经被move废弃
13.    // std::cout << "after unique ptr data is " << *p << std::endl;
14. }
```

总结

本文介绍了 `std::thread` 的基本操作，具体视频可以去B站看看我的C++视频讲解

https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101&ctype=0 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=313101&ctype=0)

代码链接

源码链接https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(6993) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(543) (JavaScript:void(0);)

热门评论

请输入正文

评论

chendaxian666

2023-09-16 13:50:10

//指针常量 char const* 指针本身不能变

这句话写错了吧，这句话也是指针内容不可变。

```
 char d = 'd';
char x = 'x';
char const* c = &d;
c = &x; //可变
*c = x; //不可变
```

♡ 喜欢(0) (JavaScript:void(0);) 回复(1) (JavaScript:void(0);)

共有1条回复, 点击查看 ⩔

没有更多评论了,点击收起 ⩓ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满...

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。
京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)
京ICP备2021033081号 (https://beian.miit.gov.cn/)